

Types on paper: an email, counted and matched

Programming Paradigms

Session 3 — warm-up, before any code is written

Exercises 1–4: in class, in pairs, about 15 minutes. Exercises 5–6: homework.

The lecture said that an algebraic data type is a closed list of shapes, written where the compiler can read it, and that the compiler pays us back by checking every `match`. We try that on the most ordinary object we own, an email message, with pen and paper. There is nothing here to run. The notation is OCaml's, and these are all we need:

<code>type t = A B C</code>	a <i>sum</i> : a value of <code>t</code> is A, or B, or C
<code>type t = A of int B of string</code>	a sum whose shapes carry something
<code>type t = { f : int; g : bool }</code>	a <i>product</i> : one value holds an <code>f</code> and a <code>g</code>
<code>int * bool</code>	a product without names, a pair
<code>t list, t option</code>	zero or more <code>t</code> ; a <code>t</code> or nothing (<code>Some x / None</code>)

1. Writing the type

Exercise 1 (the message)

Write OCaml types for the following, one line each, choosing *sum* or *product* as the facts demand. An **address** is a user name and a domain, both text. A **recipient** is an address that is either a *To*, a *Cc* or a *Bcc*. A **header** has a sender, a list of recipients, a subject, a date (take *date* as given) and an optional reply-to address. A **message** is a header and a body of text. Then draw one concrete message with two recipients as a tree: constructors at the nodes, text at the leaves.

2. Counting values

Exercise 2 (why “algebraic”)

Let `type priority = Low | Normal | High`. How many distinct values are there of each of the following types? `bool * bool`; `priority`; `priority * bool`; `priority option`; `bool option option`; `(priority * bool) option`. State the two rules that give the answers.

3. What the compiler will say

Exercise 3 (missing and unreachable)

Two functions on `recipient`, written in a hurry:

```
let address_of r = match r with
| To a -> a
| Cc a -> a

let is_hidden r = match r with
| To _ | Cc _ | Bcc _ -> false
| Bcc _ -> true
```

For each: does it compile? What does the compiler print, and what is the message called? What does `address_of (Bcc a)` do at run time, and what does `is_hidden (Bcc a)` return? Fix both.

4. States

Exercise 4 (a message's life)

A message is a draft, or has been sent at some date, or has been received at some date from some address, or has bounced with a reason. Write `type state`. Then explain why this is better than adding three fields to the header — `sent_at : date option`, `received_from : address option`, `bounce_reason : string option` — by naming one value the record version allows and the sum version cannot express.

5. A thread is a tree (homework)

Exercise 5 (replies)

A thread is a message together with the replies to it, each of which is itself a thread: `type thread = Msg of message * thread list`. Write, on paper, `count : thread -> int`, the number of messages in a thread. It needs a second function on `thread list`; write both, and say why each recursive call is on something smaller.

6. Kinds and operations (homework)

Exercise 6 (the table)

Three operations on `recipient`: `show` (as text), `deliver` (send to its address), `is_hidden`. Draw the table with the three kinds `To`, `Cc`, `Bcc` as rows and the operations as columns. In OCaml, which cells sit together in one function? Now add a kind, `Bcc_group` of `address list`, and separately add an operation, `count`: for each change, which cells are new and how many existing functions are touched? Sketch the same type as a Java abstract class with three subclasses and answer the same two questions.